# Turning the Valve

## Diving into the Steam Platform

Bryan Alexander
October 2026

# whoami

- Research Science Director @ Atredis Partners
- former red team, exploit dev, EDR enthusiast



- dronesec.net
- @droner.bsky.social
- @drone@infosec.exchange
- atredis.com

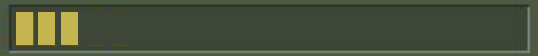
# Steam?

- Digital distribution platform for PC games
- Originally released 2003
- Holds approximately 75% of market share

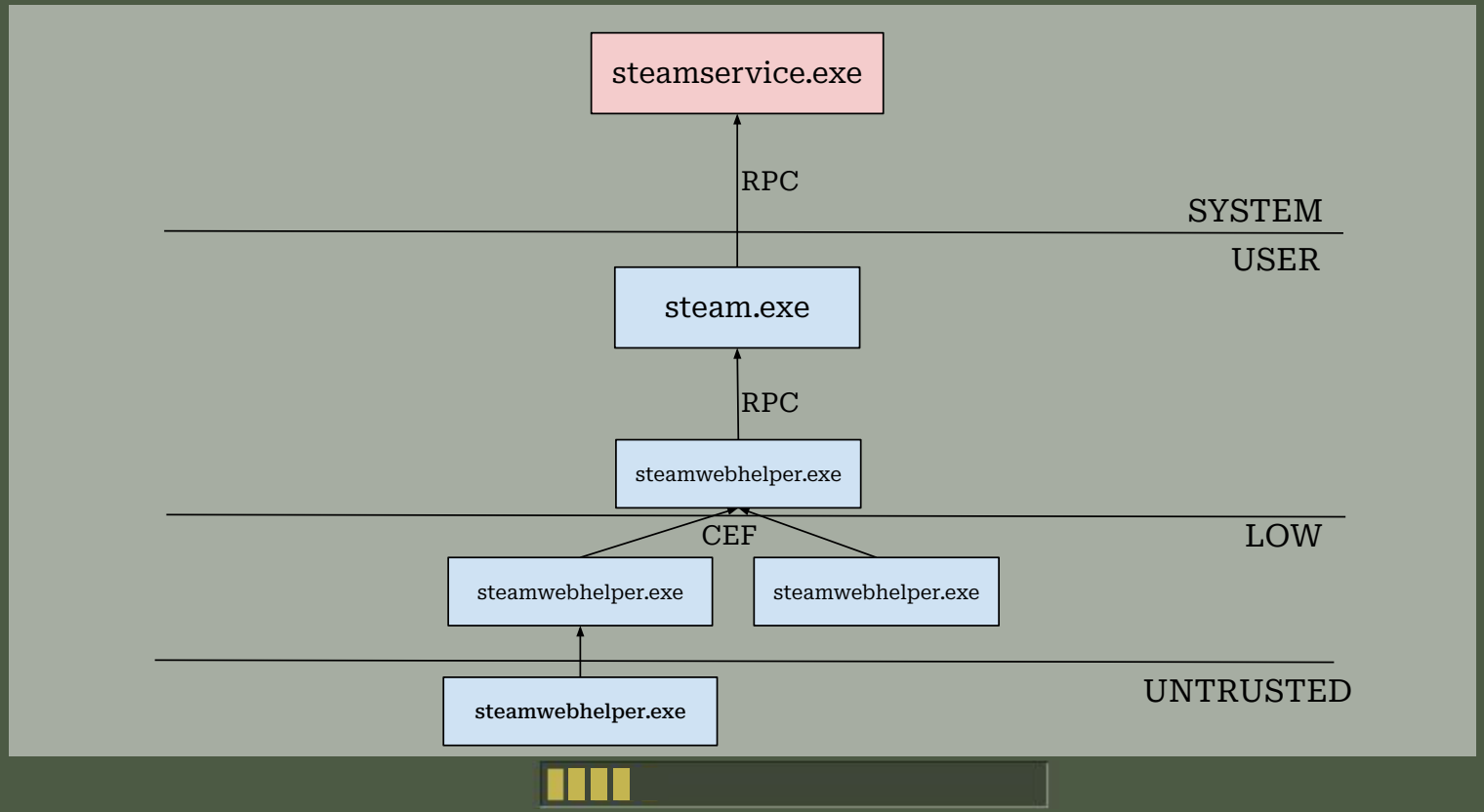


# Agenda

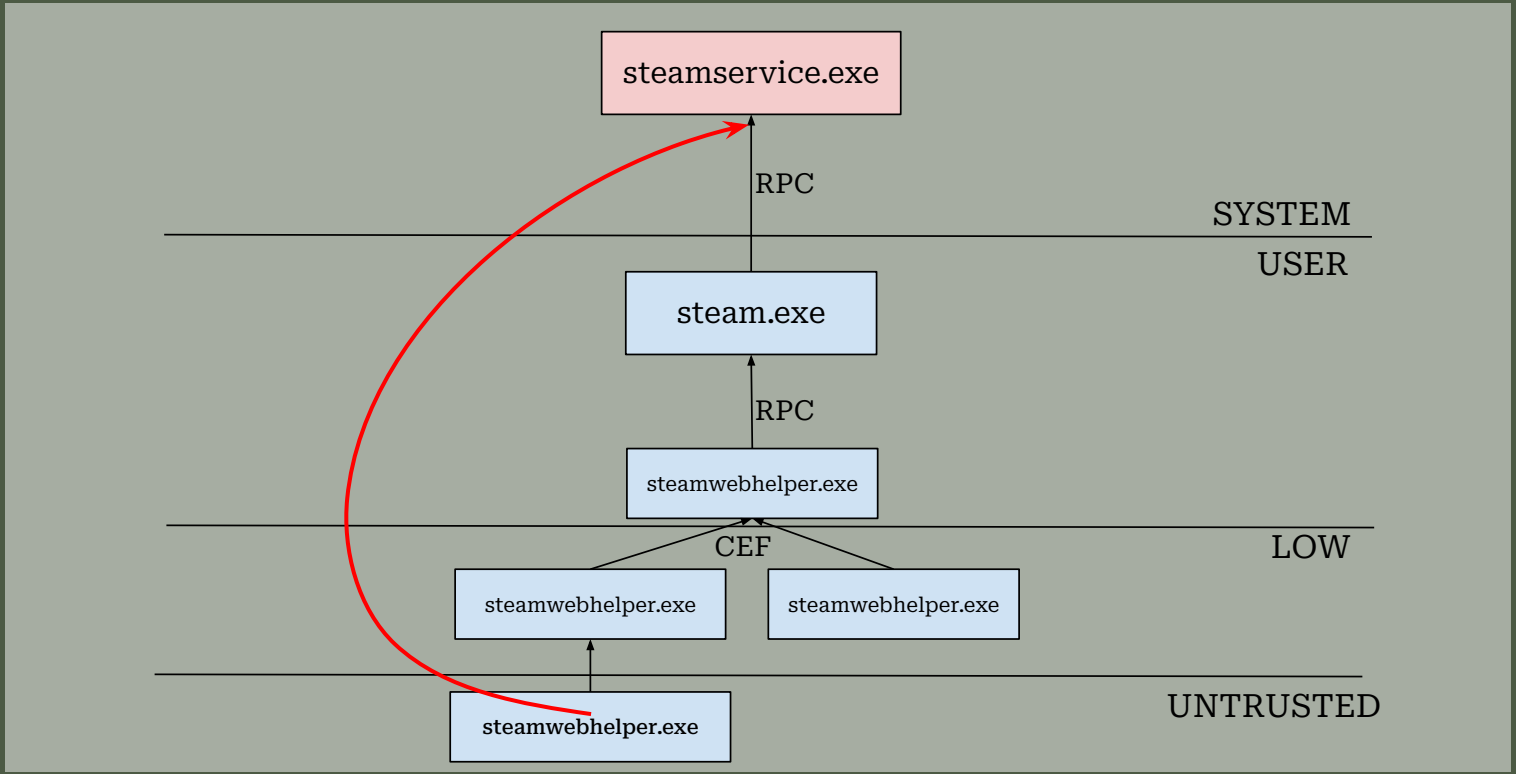
- architecture
- bugs
  - Local
  - Remote
- conclusion



# Architecture - Steam

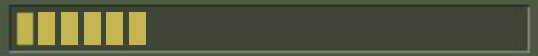


# Architecture - Steam



# Architecture - Steam

- IPC endpoint supports N connections from *any* IL
- How do we connect?



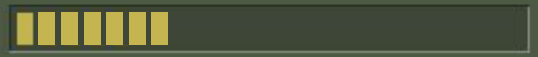
# Architecture - Steam

Global\SteamClientService_SharedMemFil

| | |
|---|---|
| State | k_EMasterStateAvailable |
| Server PID | 8111 |
| Client PID | |
| Read Pipe | |
| Write Pipe | |
| Client Event | |
| Notification Event | |

steam.exe

steamservice.exe



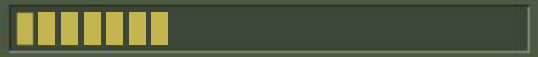
# Architecture - Steam

Global\SteamClientService_SharedMemFil

| | |
|---|---|
| State | k_EClientRequesting |
| Server PID | 8111 |
| Client PID | 9999 |
| Read Pipe | |
| Write Pipe | |
| Client Event | 0x40566790 |
| Notification Event | |

steam.exe

steamservice.exe



# Architecture - Steam

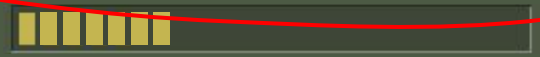
Global\SteamClientService_SharedMemFil

| | |
|---|---|
| State | k_EServerResponded |
| Server PID | 8111 |
| Client PID | 9999 |
| Read Pipe | 0x40566795 |
| Write Pipe | 0x4056670a |
| Client Event | 0x40566790 |
| Notification Event | 0x4056770f |

steam.exe

steamservice.exe

```
new CCrossProcessPipe
CreatePipe()
DuplicateHandle(..)
```



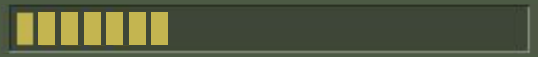
# Architecture - Steam

Global\SteamClientService_SharedMemFil

| | |
|---|---|
| State | k_EClientAcknowledged |
| Server PID | 8111 |
| Client PID | 9999 |
| Read Pipe | 0x40566795 |
| Write Pipe | 0x4056670a |
| Client Event | 0x40566790 |
| Notification Event | 0x4056770f |

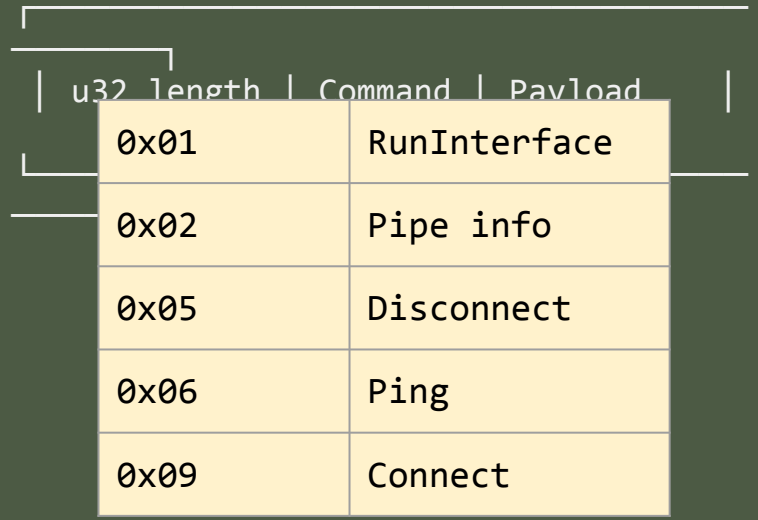
steam.exe

steamservice.exe

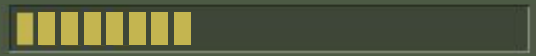


# Architecture - Steam

- All messages exchanged via the two anonymous pipes
  - Short connect exchange
- Each packet is length-prefixed
- Five dispatch types
- What’s available?
  - 7 interfaces and approximately 90 functions

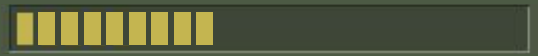



| | |
|---|---|
| 0x01 | RunInterface |
| 0x02 | Pipe info |
| 0x05 | Disconnect |
| 0x06 | Ping |
| 0x09 | Connect |

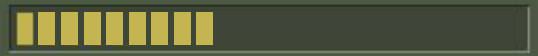


# bugs

- Local
  - IClientModuleManager
  - IClientInstallUtils
  - Secure Desktop Capture
  - BYOVD
  - Vulkan Registry Injection
- Remote
  - RecvMsgDownloadScheduleChanged
  - RecvMsgAppStatus
  - Shortcut Injection
  - …



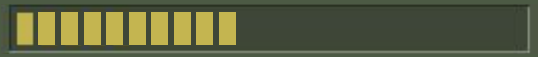
Local bugs



# bugs - IClientModuleManager

- Interface that manages DLLs loaded into the service process
- Supports dynamic invocation of functions within those DLLs provided the DLL is digitally signed by Valve
- Except in legacy cases!

```
bool LoadModule(int nModuleID, const void *pData, uint32_t cbData)
uint32_t CallFunction(int nModuleID, int nArg2, int nArg3,
                      const void *pIn, uint32_t cbIn,
                      void *pOut, int cbOut, void *pOut2)
```



# bugs - IClientModuleManager

- When CallFunction is called with FLAG_CHILD_PROCESS, CreateProcess is invoked on %SteamInstallDir%\legacycompat\steamservice.exe
- Called with /runfunc to invoke an exported function *after* RSA signature check
- \legacycompat\ is writable!

```
bool LoadModule(int nModuleID, const void *pData, uint32_t cbData)

uint32_t CallFunction(int nModuleID, int nArg2, int nArg3,
                      const void *pIn, uint32_t cbIn,
                      void *pOut, int cbOut, void *pOut2)
```



# bugs - IClientInstallUtils

- Large interface that’s functionally overloaded
- 24 different endpoints, supporting everything from firewall configuration to game and resource management
- One in particular caught my attention

```
IClientInstallUtils::AddInstallScriptToWhitelist(String path_to_vdf,
                                                 String installDir);
```



# bugs - IClientInstallUtils

- Valve Data File (VDF) are instrumental opaque text files used to configure and manage game and dependency installation

```
"installscript"
{
  "utf8_registry_strings"   "1"
  "registry"
  {
    "hkey_local_machine\\software\\valve\\cs2"
    {
      "string"
      {
        "installpath"   "%INSTALLDIR%"
      }
    }
  }
}
"kvsignatures"
{
  "installscript"   "8a80c9a3a8ab75b8f133929..."
}
```

# bugs - IClientInstallUtils

- `IClientInstallUtils::RunInstallScript` will parse and execute the provided VDF
- Various actions are supported, including running processes, but only *after* signature verification
- An allow list is used to manage allowed paths






# bugs - IClientInstallUtils

- Remember the installPath argument? It fully resolves %INSTALLDIR% variables in validated VDFs

```
"Run Process"
{
  "Visual C++ 2010 SP1 Redistributable Package (x86)"
  {
    "process 1"   "%INSTALLDIR%\\redist\\vcredist_x86.exe"
    "command 1"   "/q:a"
    "description"   "Visual C++ 2010 SP1 Redistributable
Package (x86)"
    "NoCleanUp"   "1"
    "IgnoreExitCode"    "1"
  }
}
```

# bugs - IClientInstallUtils

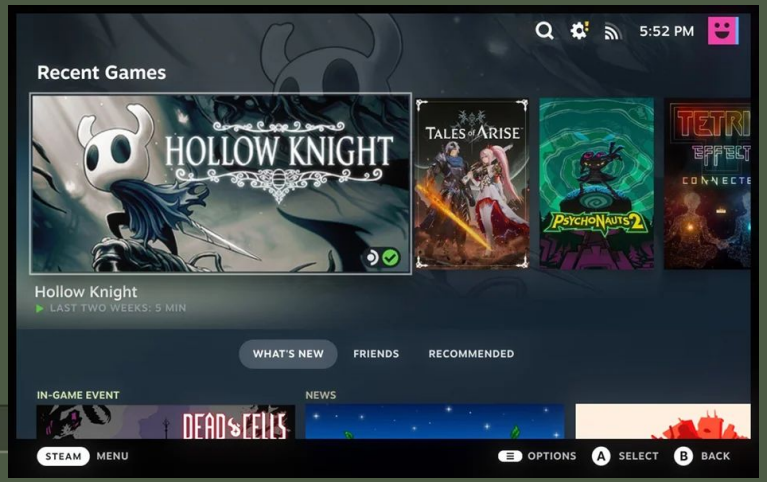
- Grab any VDF with a process block and %INSTALLDIR% target

```
AddInstallScriptToWhiteList("C:\Temp\redist\valid.vdf",
                            "C:\Temp");
copy my_payload.exe C:\Temp\redist\vcredist_x86.exe
RunInstallScript("C:\Temp\redist\valid.vdf", 0, 0);
```



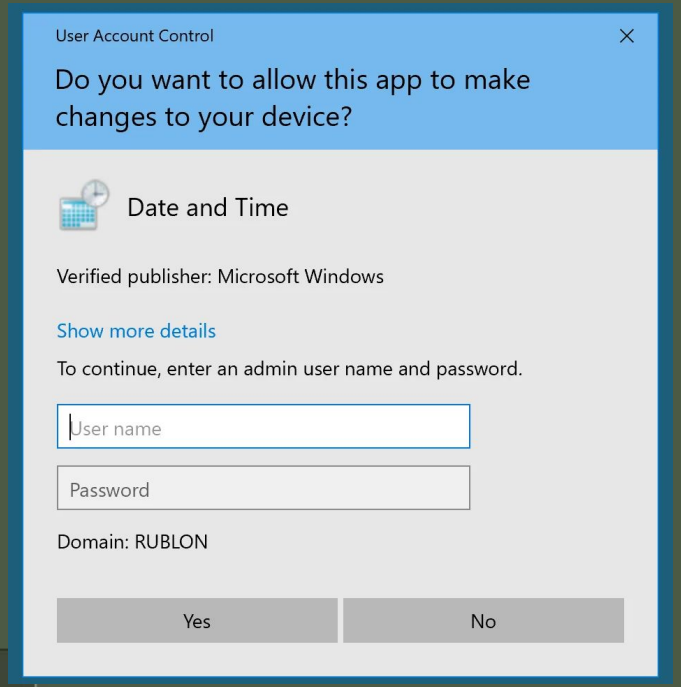
# bugs - “Secure” Desktop

- Steam supports various gaming modes that don’t necessarily include a user at the PC
- Remote Play can stream to different machines (either PCs or televisions)
- Big Picture provides console-like experience on a PC

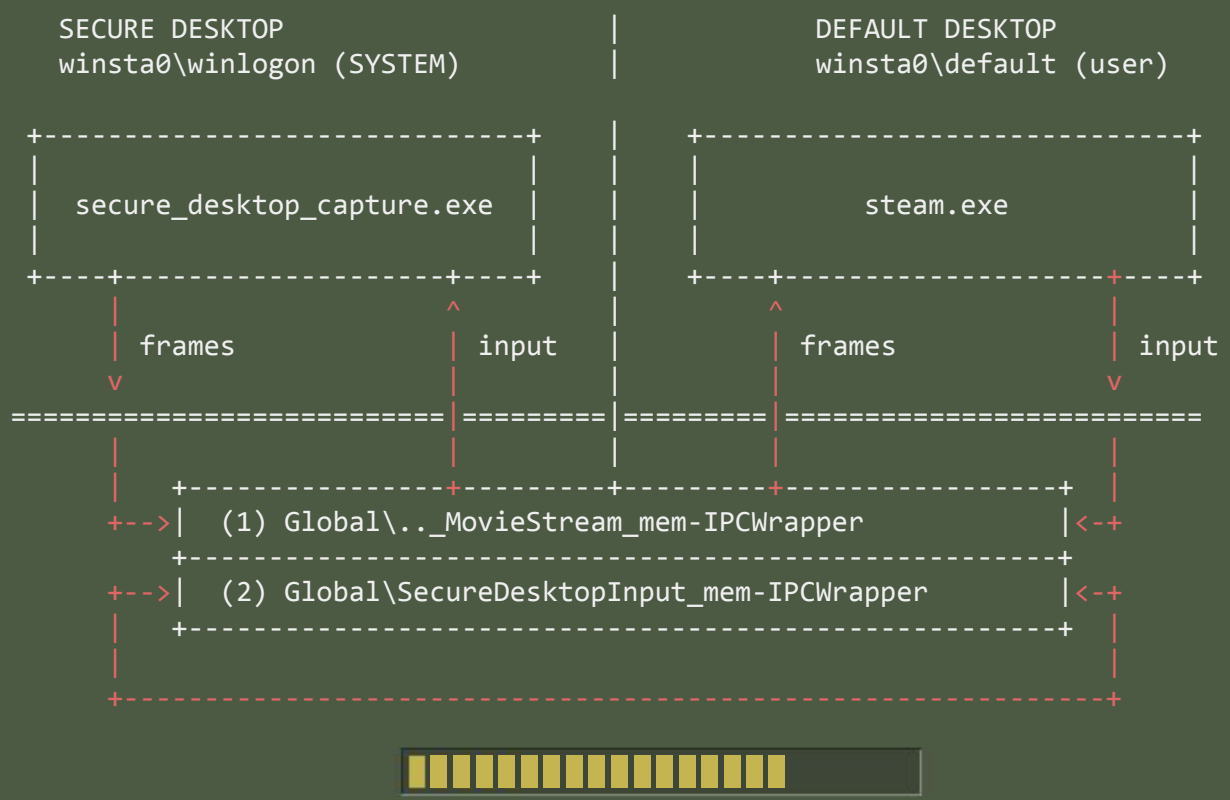


# bugs - “Secure” Desktop

- What happens when the install process requires elevation?
- Steam needs to handle UAC popups



# bugs - “Secure” Desktop



# bugs - “Secure” Desktop

- Users provide blanket authorization for 90 days (stored in HKLM)
- Spawn a process that requires administrator
- Spam VK_Y into shared input buffer



Remote bugs



# SteamToSteam

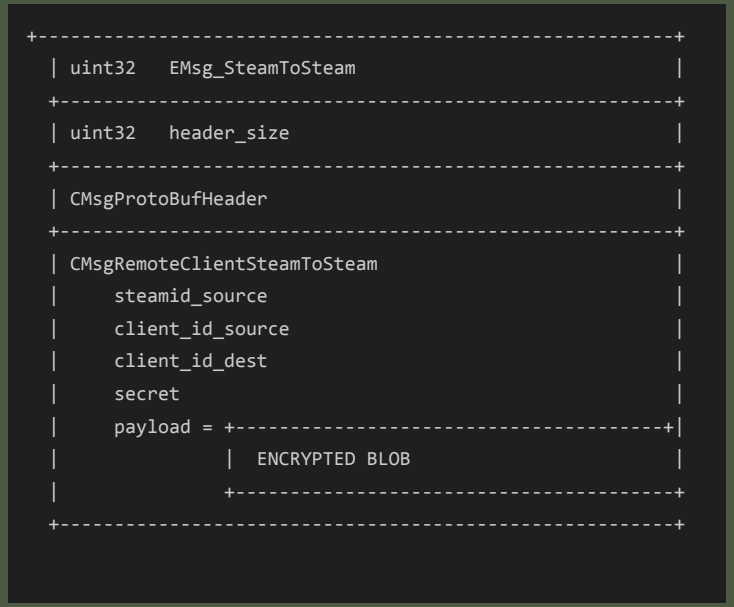
- When a user logs into their Steam account, state is synchronized with every other authenticated instance
- Each instance maintains a list of all connected clients
- Valve’s cloud CM router handles message delivery

```
struct CRemoteClient {
    uint32_t  m_ClientID;
            /* ... */
    void     *m_pPendingChore;
    uint8_t   m_bTrusted;
            /* ... */
    CUtlMap   m_mapAppInstallState;
            /* ... */
    CUtlMap   m_mapShortcutRemap_A;
    CUtlMap   m_mapShortcutRemap_B;
            /* ... */
    uint64_t  m_hPendingJob;
 };
```



# SteamToSteam

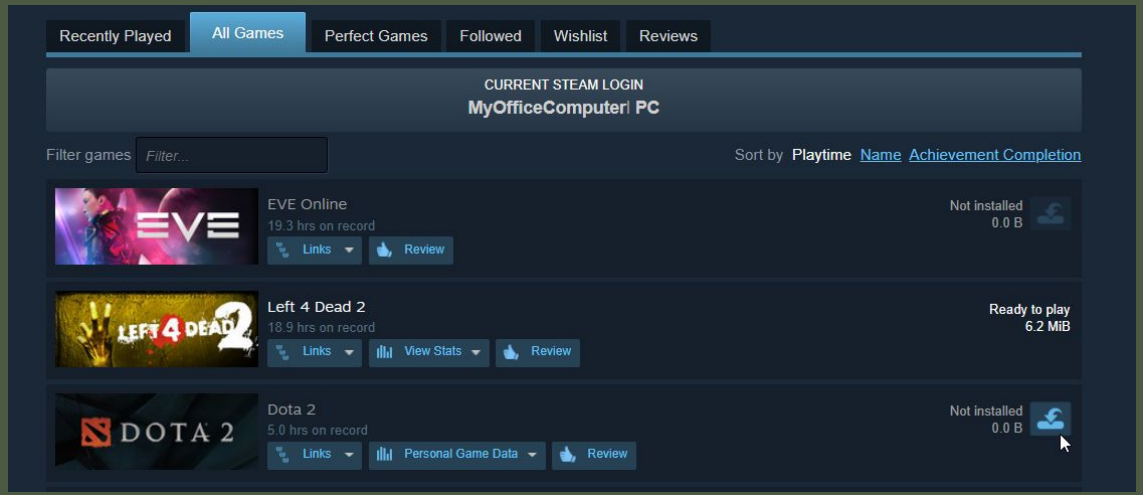
- Messages are protobuf and encrypted with a shared, global session key generated by Valve
- An outer structure carries the packet from a cloud router to the client destination
- While there are nearly 2000 EMsg types, only 25 or so apply to SteamToSteam






# RecvMsgDownloadScheduleChanged

- Steam supports remote download management






# RecvMsgDownloadScheduleChanged

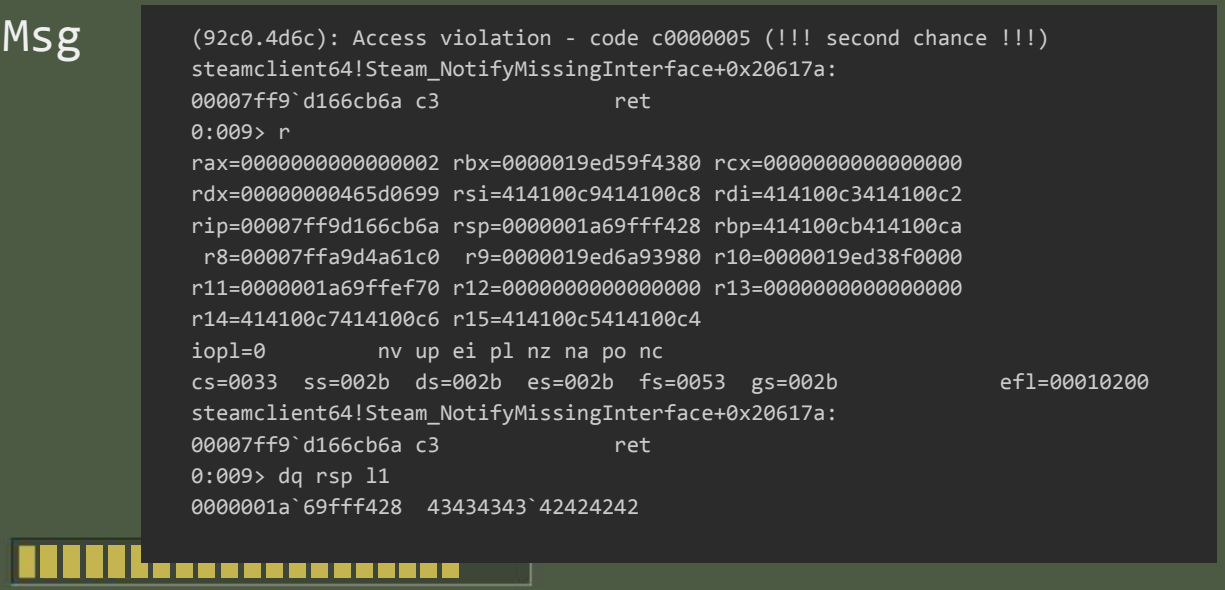
- Schedules are shared via SteamToSteam packets
- When a schedule changes, EMsg 9526 is emitted
- apps_scheduled is a list of app-id's in the download queue
- The list is copied out into a fixed-sized buffer without bounds checking

```
message
CMsgRemoteClientDownloadScheduleChanged {
  optional bool   download_enabled    = 1;
  optional uint32 start_index         = 2;
  optional bool   final_message       = 3;
  optional uint32 num_apps_scheduled  = 4;
  repeated uint32 apps_scheduled      = 5;
  }
```



# RecvMsgDownloadScheduleChanged

- To trigger, login to an account with multiple sessions
- Inject into steam.exe and re-use networking utilities to send malformed EMsg



# Remotely Exploiting the EMsg

- No CFG or stack canaries
- All modules compiled with ASLR
- We need a leak!



# Remotely Exploiting the EMsg

- When crafting CMsgRemoteClientDownloadScheduleChanged messages, notifications are copied directly into a stack buffer
- This message is emitted both on startup and on schedule change
- Stack buffer is never initialized and always transmitted!

```
void BuildNotification() {
    _DWORD v17[38]; // [0]
    int    v16 = 0;
    ...
    for ( v4 = head; v4 != -1; v4 = next ) {
        v17[v5] = scheduled_app_id;
        v5 = ++v16;
        if ( v5 == 32 && more_entries ) {
            ... ; sub_138900E40(mgr, &struct); // [1]
            v5 = 0; v16 = 0;
        }
    }
}

void sub_138900E40(LPVOID mgr, LPVOID struct){
    ...
    *(_DWORD *)(msg + 48) = a2[12];
    v12 = a2 + 16;  v14 = v12 + 32;
    do { add_repeated_uint32(msg, *v12); ++v12; } while ( v12
!= v14 );   // [2]
    ...
}
```

# Remotely Exploiting the EMsg

- Listen for `CMsgRemoteClientDownloadScheduleChanged` messages and parse out the 32 DWORDs
- Reconstruct base address for a host DLL (steamclient64.dll)
- CreateProcess ROP payload



# Remotely Exploiting the EMsg

Demo



# Disclosure

- All bugs disclosed to Valve via HackerOne (with varying degrees of success..)
- 2x duplicates
- 1x collision with another researcher (who disclosed)
- All bugs patched and/or reached 90 days
- Full blog, with tooling and PoCs, next week!



Thank you!

